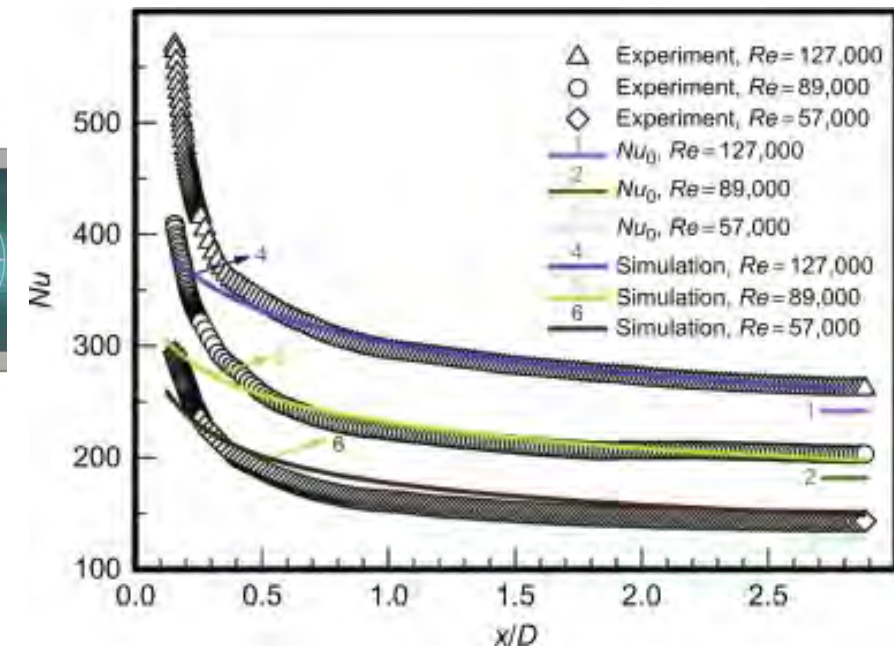
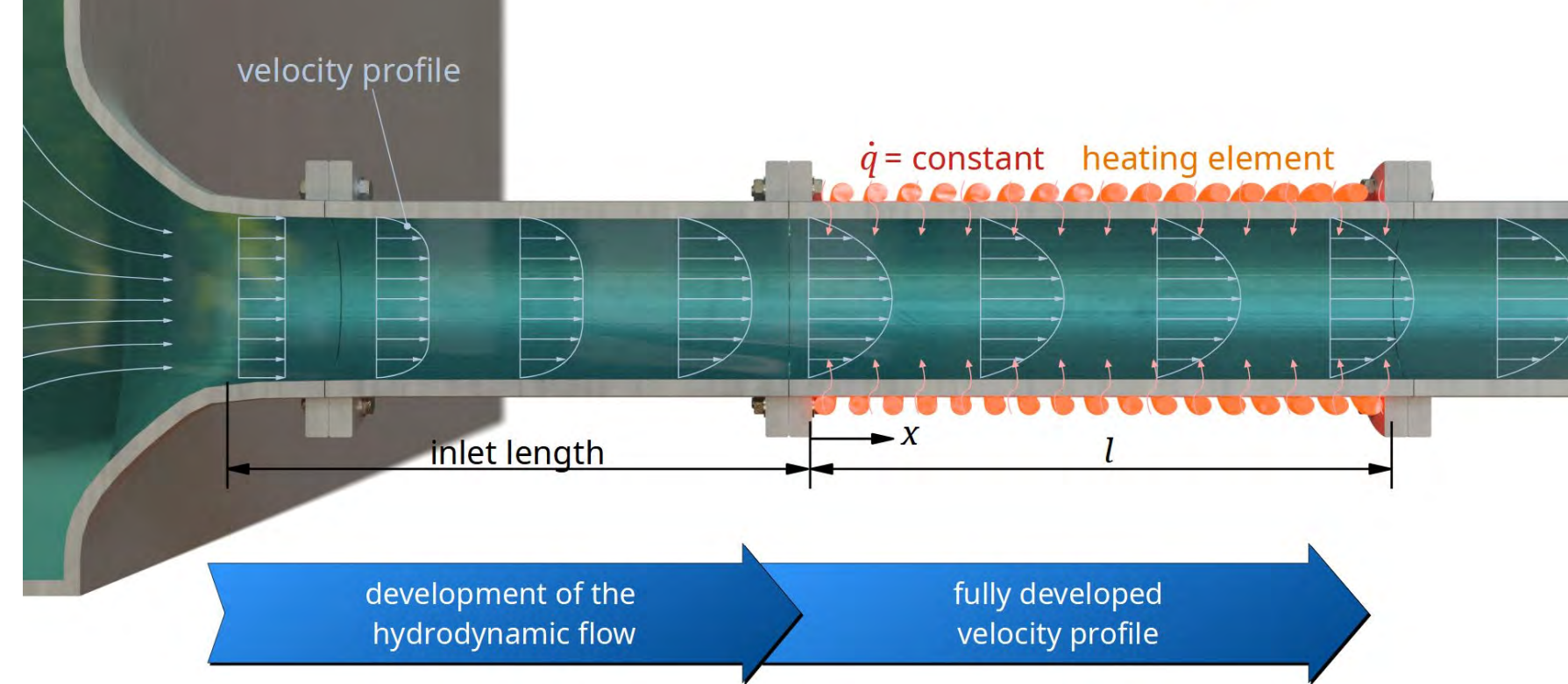


ENM 5310: Data-driven Modeling and Probabilistic Scientific Computing

Lecture #0: Introduction, motivation and course logistics

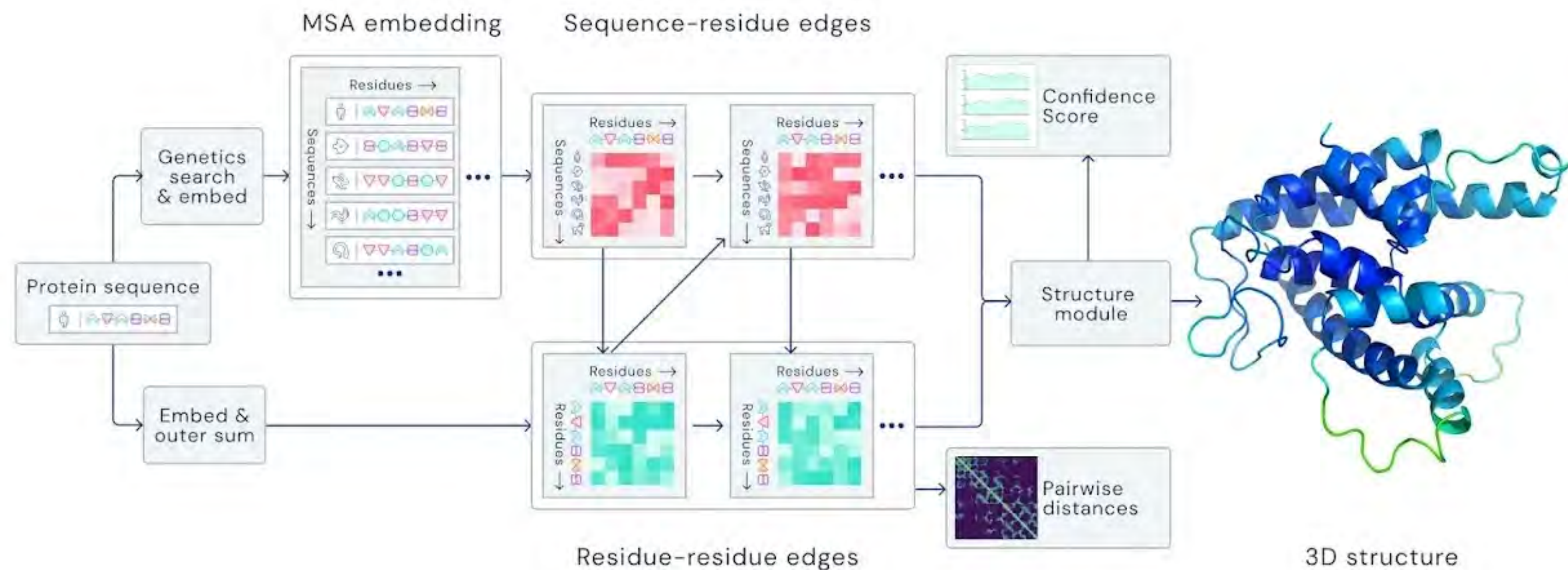




$$Nu = 0.023 \cdot Re^{0.8} Pr^{0.4}$$

Dittus-Boelter equation: an empirical correlation used in fluid mechanics and heat transfer to calculate the Nusselt number (Nu) for turbulent fluid flow inside a pipe or tube.

- **Nu (Nusselt Number):** The ratio of convective heat transfer to conductive heat transfer at the fluid boundary. Solving for Nu allows you to calculate the actual heat transfer coefficient (h) of the system.
- **Re (Reynolds Number):** A dimensionless number that determines whether the fluid flow is laminar or turbulent by measuring the ratio of inertial forces to viscous forces.
- **Pr (Prandtl Number):** A dimensionless number representing the ratio of momentum diffusivity (viscosity) to thermal diffusivity.



Data-driven Modeling

Data-driven modeling is the construction of mathematical models whose:

- *Structure chosen by us* — where physics and inductive bias enter (e.g. Dittus–Boelter’s power-law form was a choice).
- *Content determined by data* — estimation (e.g. three coefficients)

ENM5310 is about:

- *How to build such models?* We will discuss the mathematical and computational underpinnings of modern machine learning.
- *How much to trust them?* This part most courses drop and this one will not. For example, Dittus–Boelter has a validity envelope, and nothing in the correlation tells you when you have left it.

I. predict	II. compress	III. decide
supervised	unsupervised	sequential / active

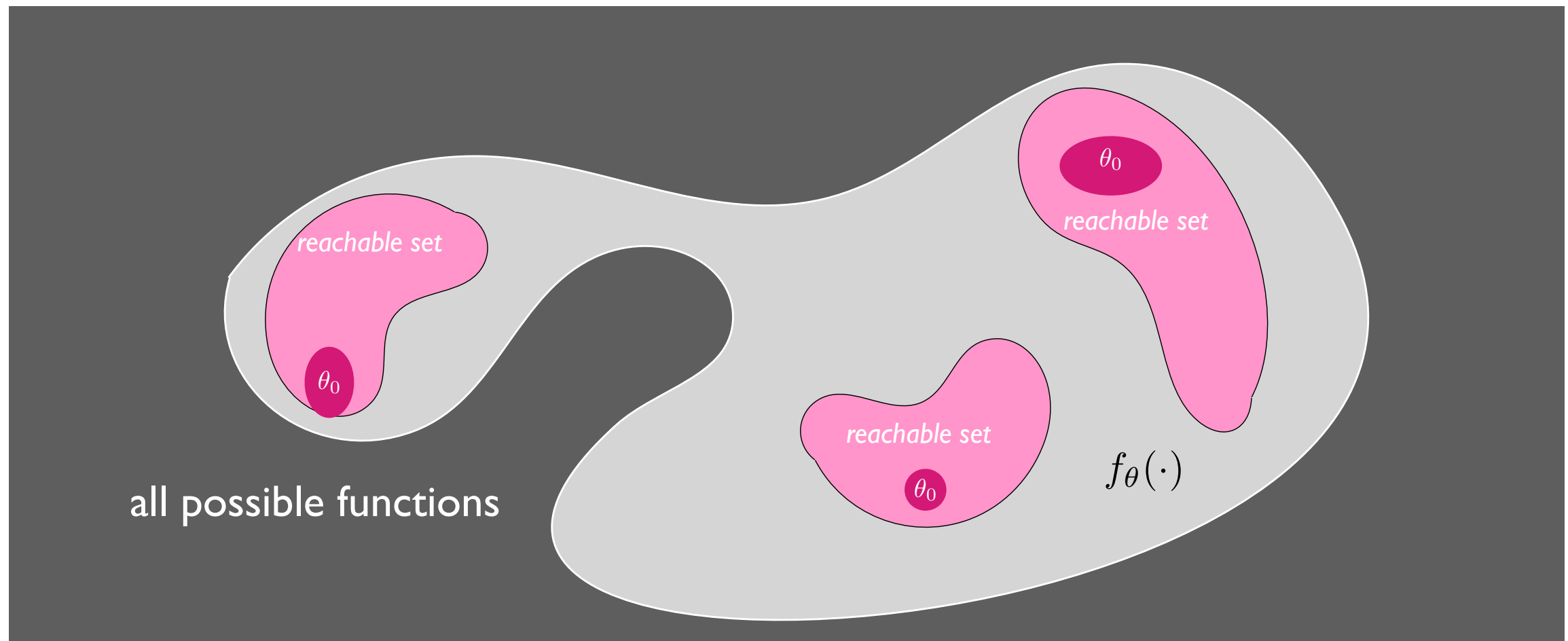
Everything on the syllabus is one of these three problems, plus the shared machinery for solving them. If at any point in the semester you cannot tell me which column we are in, stop me!

Supervised learning

Given $\{(x_i, y_i)\}_{i=1}^n \sim p(x, y)$, find $f : \mathcal{X} \rightarrow \mathcal{Y}$ minimizing

$$\mathbb{E}_{(x,y) \sim p} [\ell(y, f(x))]$$

Supervised learning



Key elements:

- Architecture/inductive bias
- Initialization
- Training data
- Normalization/Non-dimensionalization
- Learning objective
- Learning algorithm / Optimization

Supervised learning

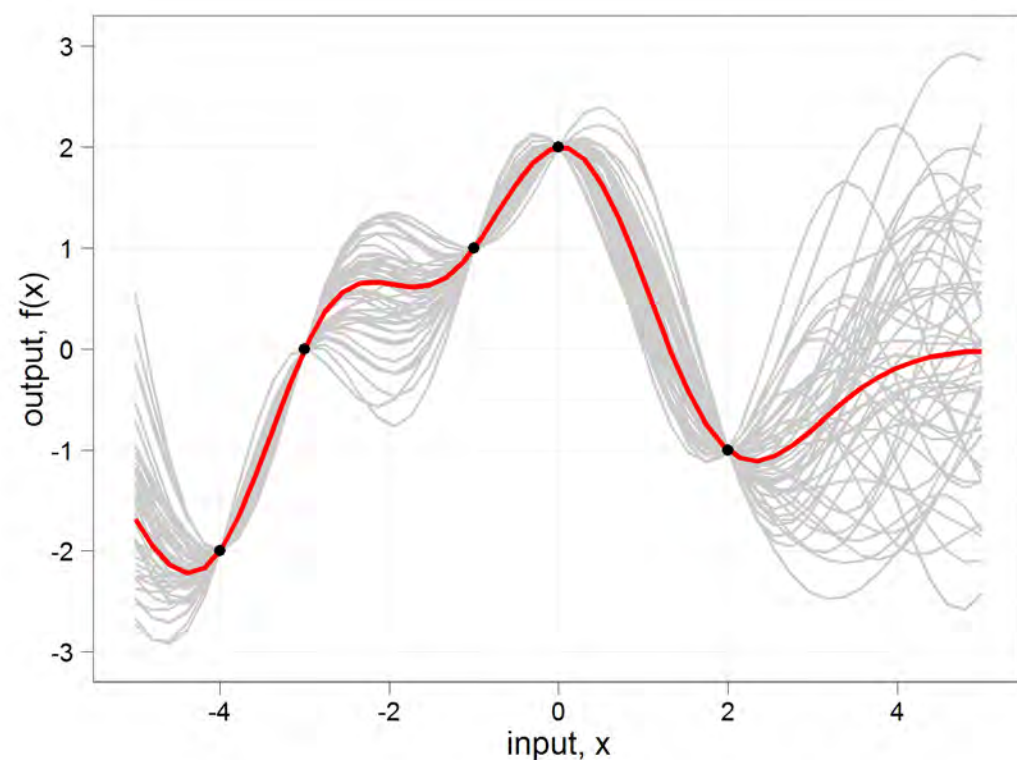
$$f : \mathcal{X} \rightarrow \mathcal{Y}$$

$$\mathcal{D} = \{x, y\}, \quad x \in \mathcal{X}, \quad y \in \mathcal{Y}$$

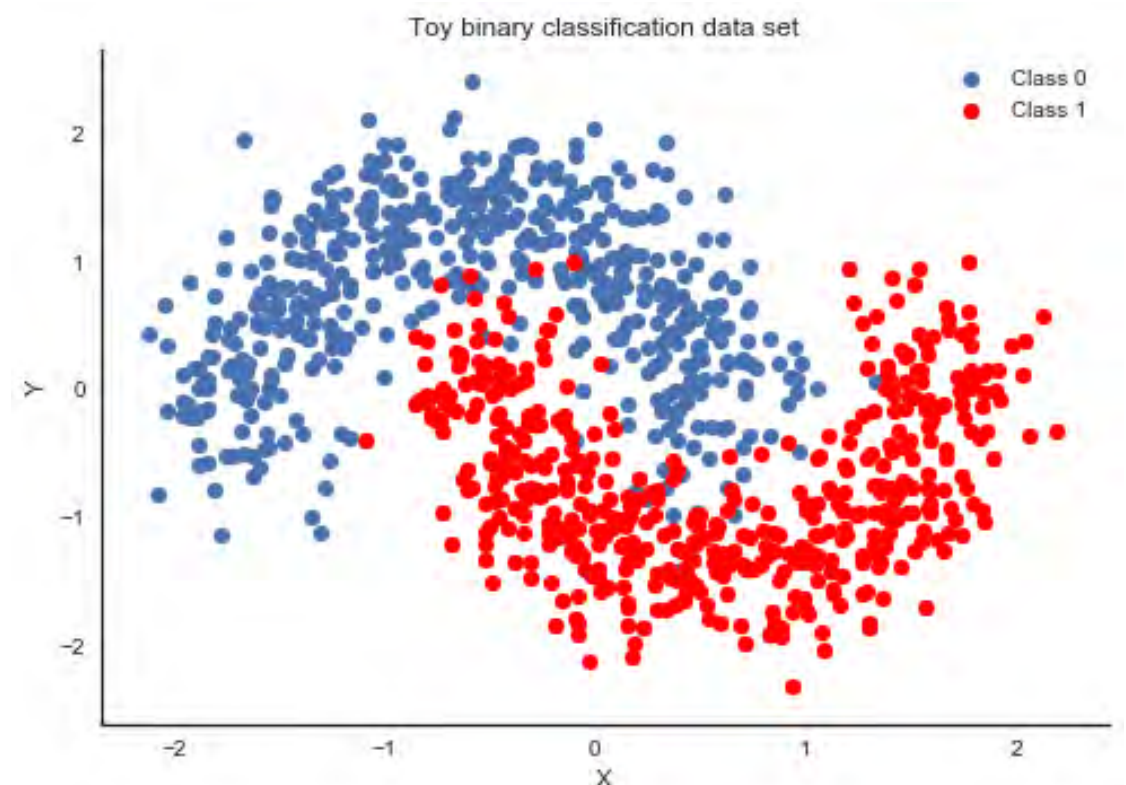
$$y = f(x) + \epsilon$$

$$p(f(x^*)|x^*, \mathcal{D})$$

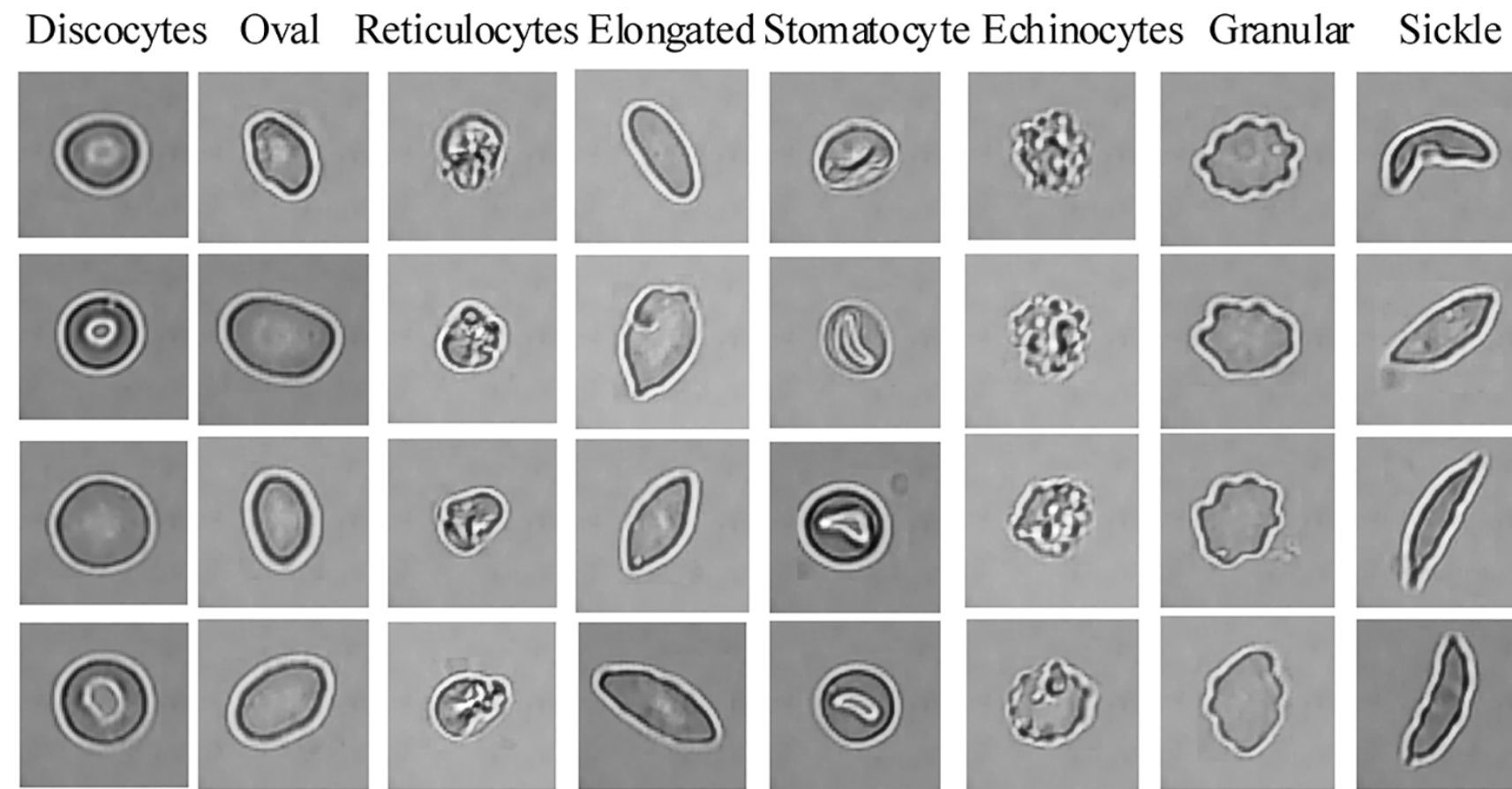
Regression



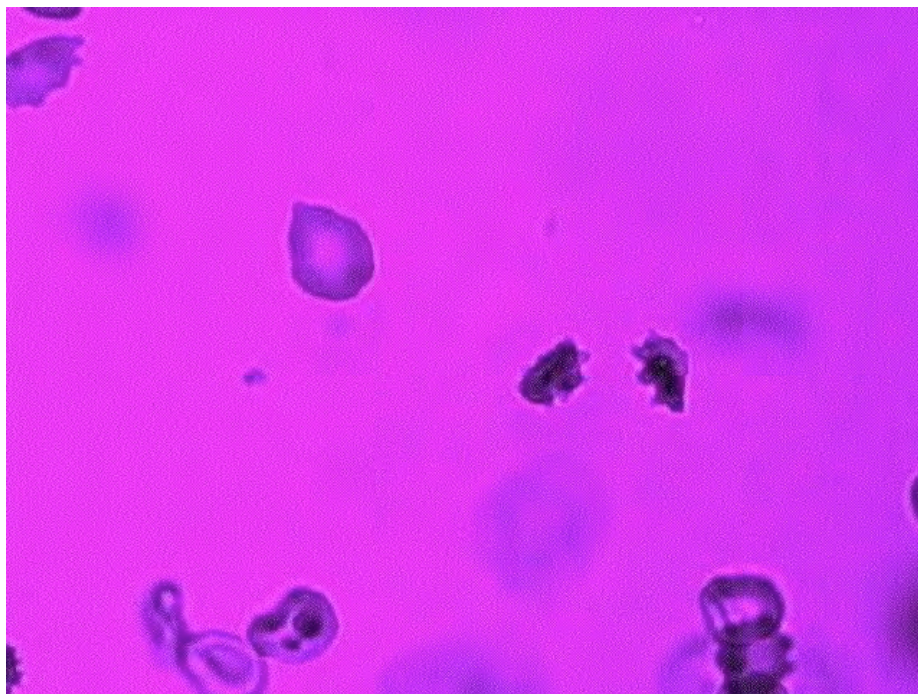
Classification



Classification



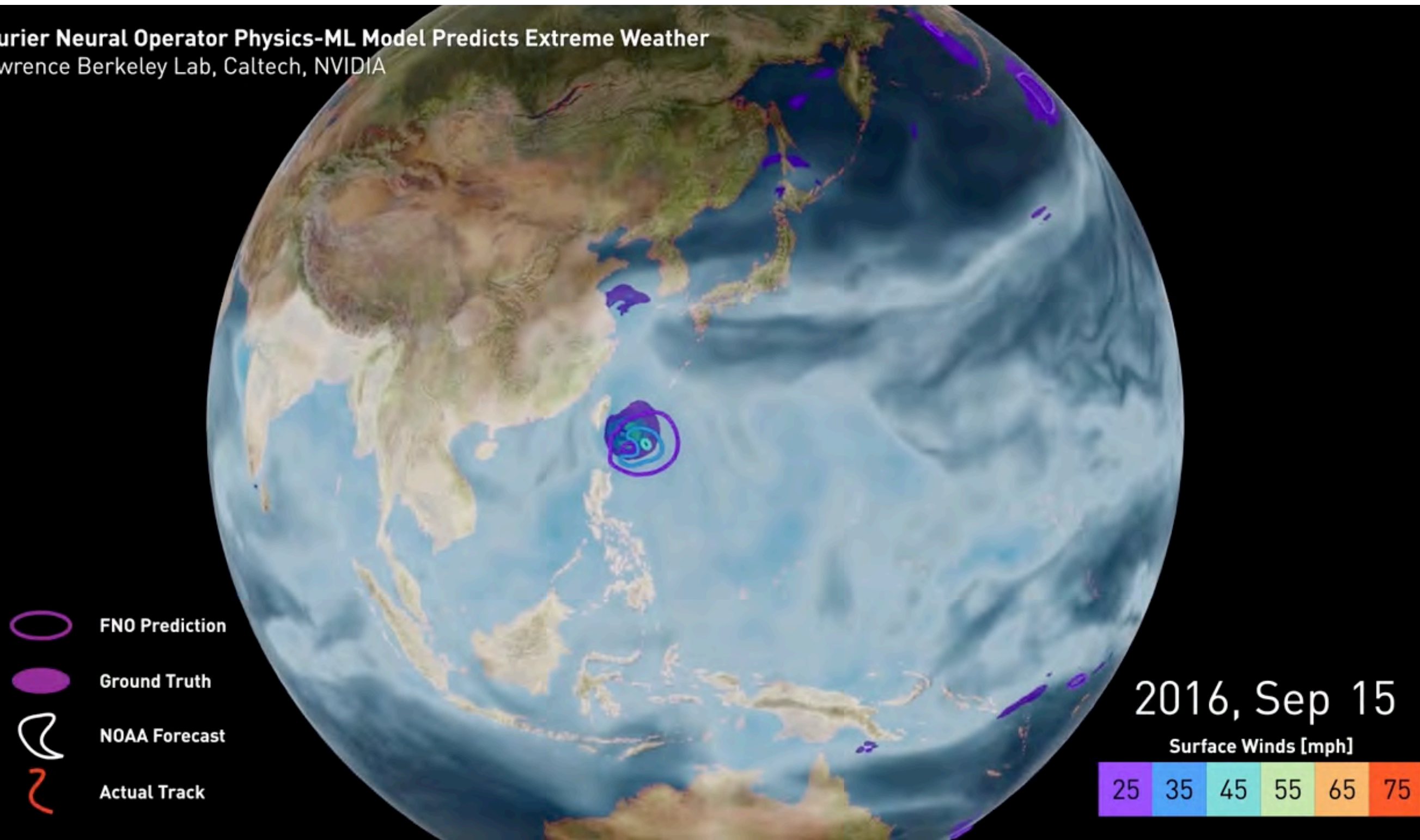
$$\mathcal{D} = \{x, y\}, \quad x \in \mathcal{X}, \quad y \in \mathcal{Y}$$



$$y = f(x) + \epsilon$$

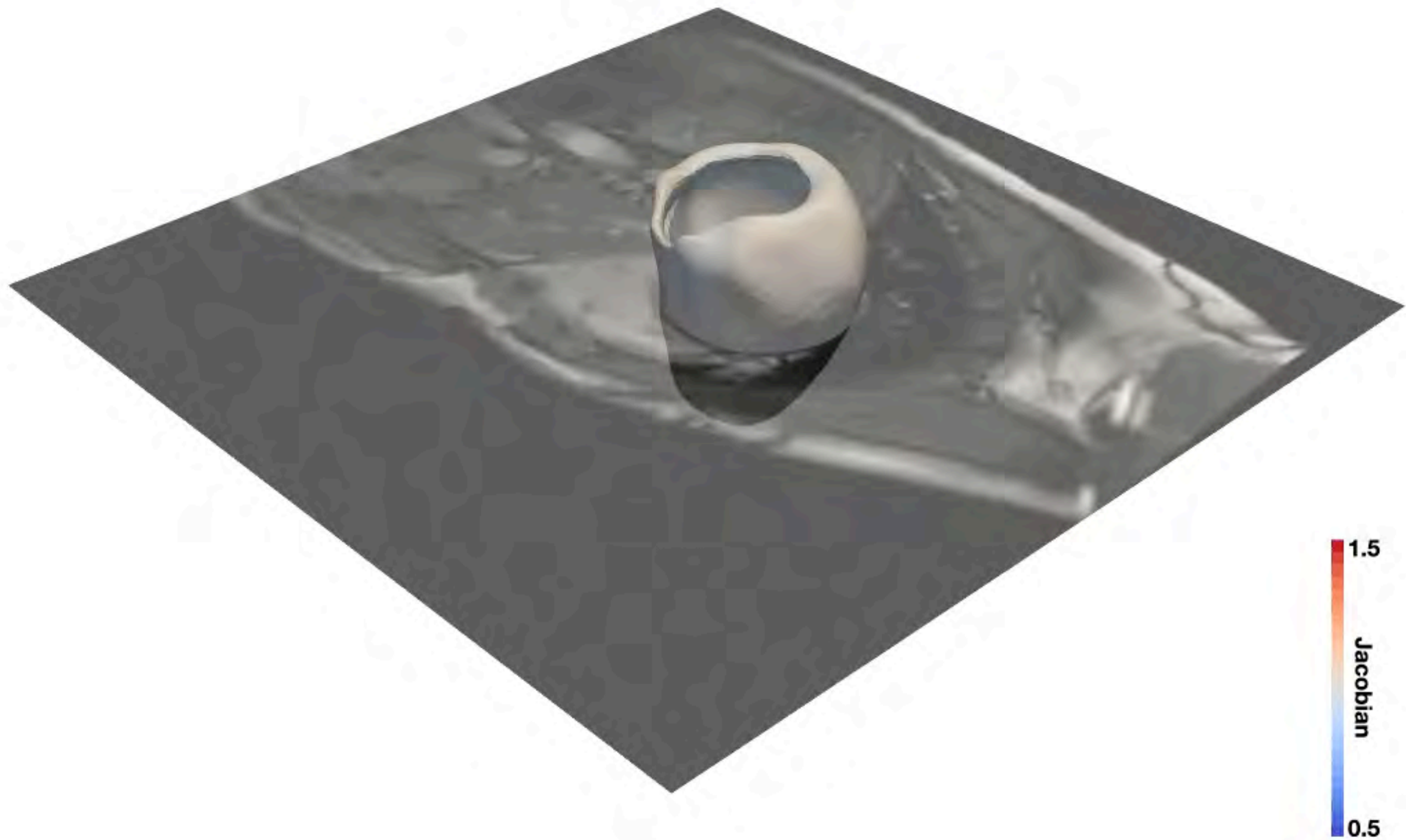
Spatio-temporal prediction

Fourier Neural Operator Physics-ML Model Predicts Extreme Weather
Lawrence Berkeley Lab, Caltech, NVIDIA



<https://blogs.nvidia.com/blog/2021/11/16/ai-science-climate-change/>

Self-supervised Learning

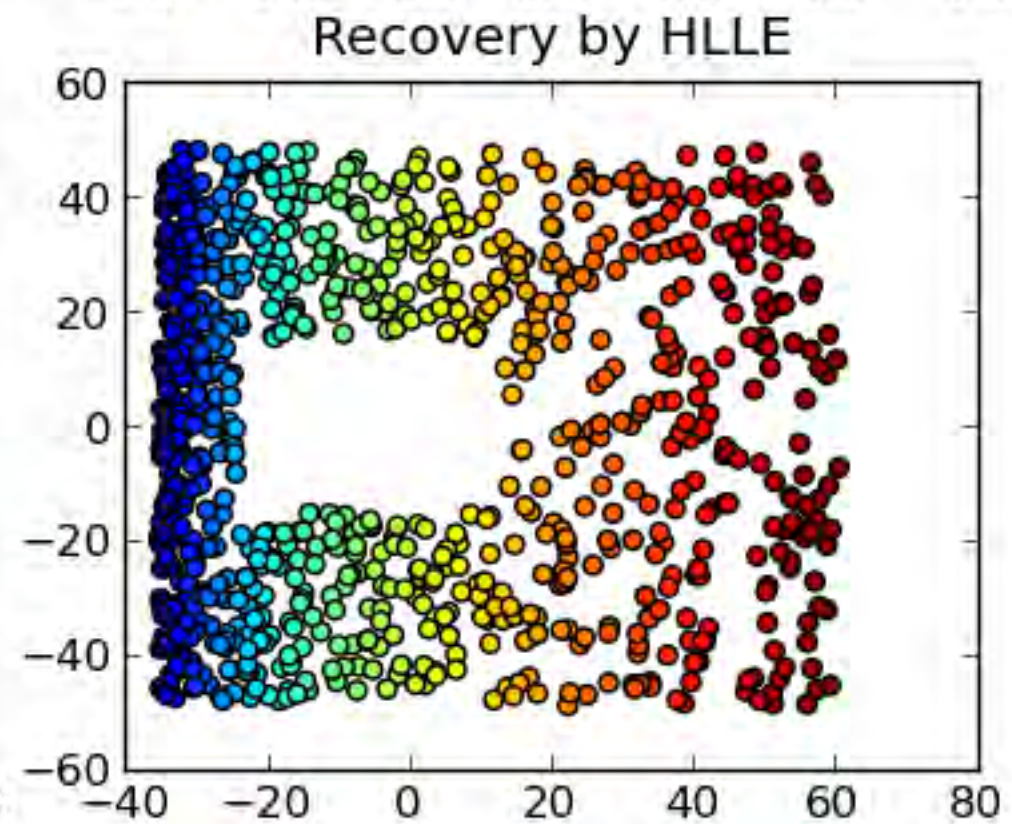
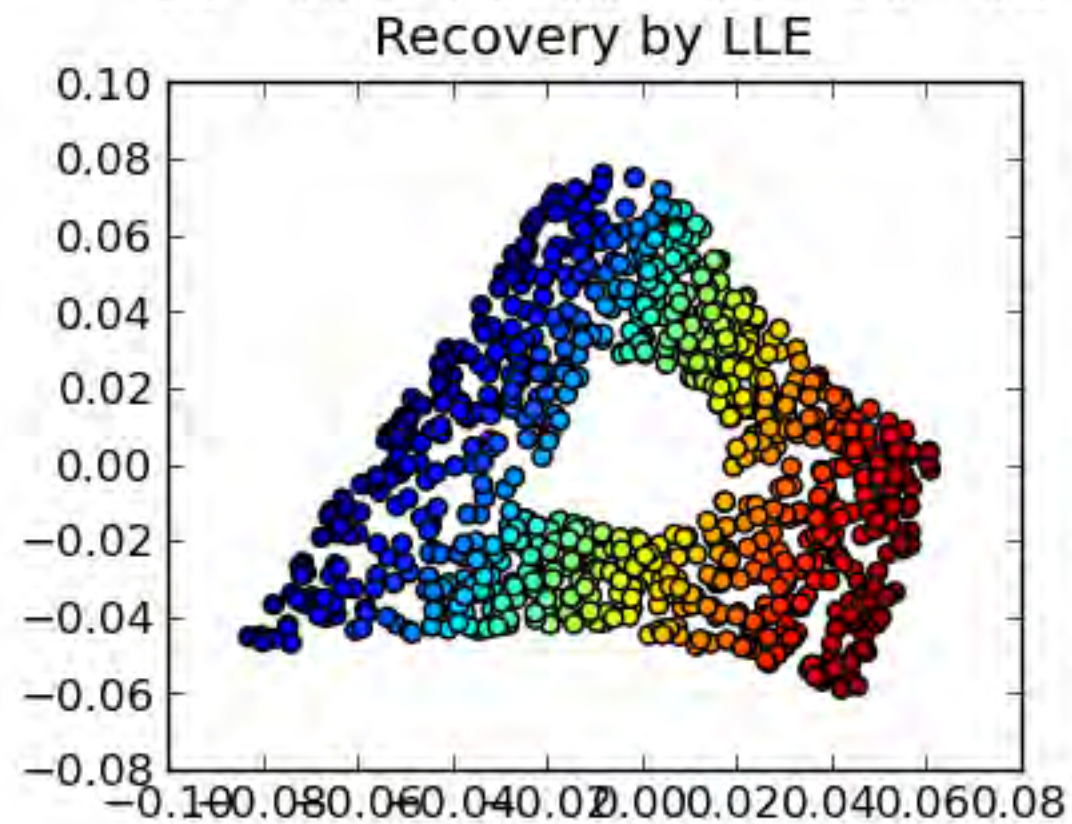
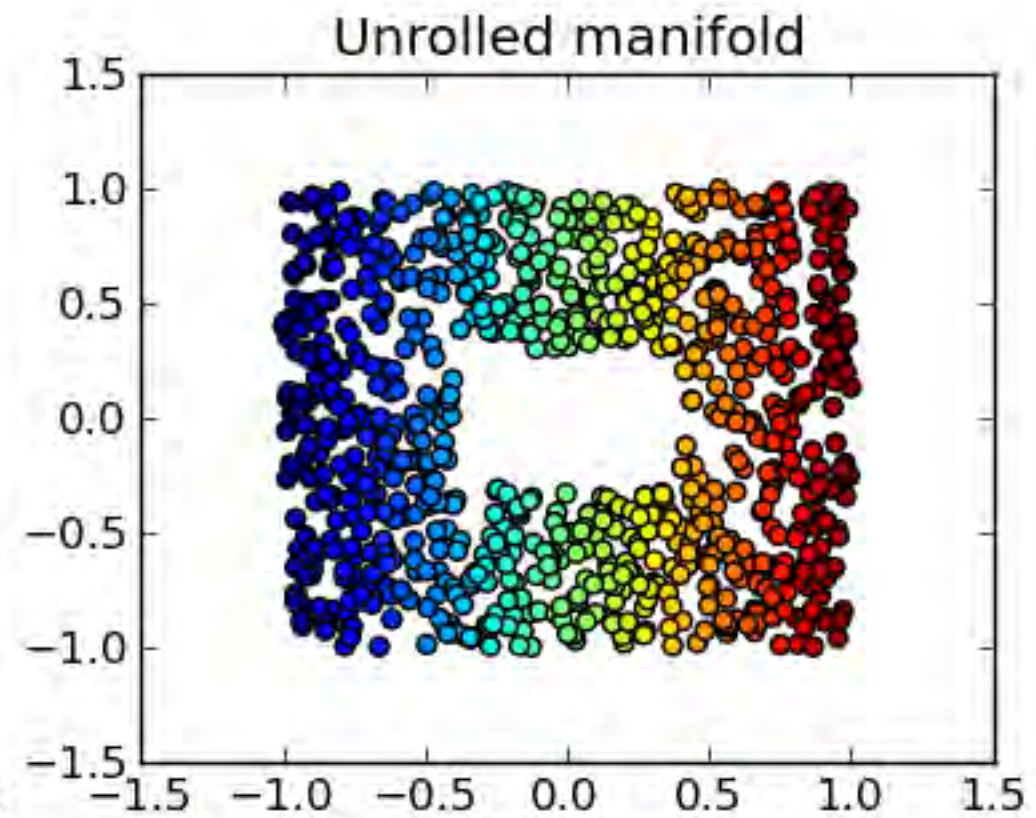
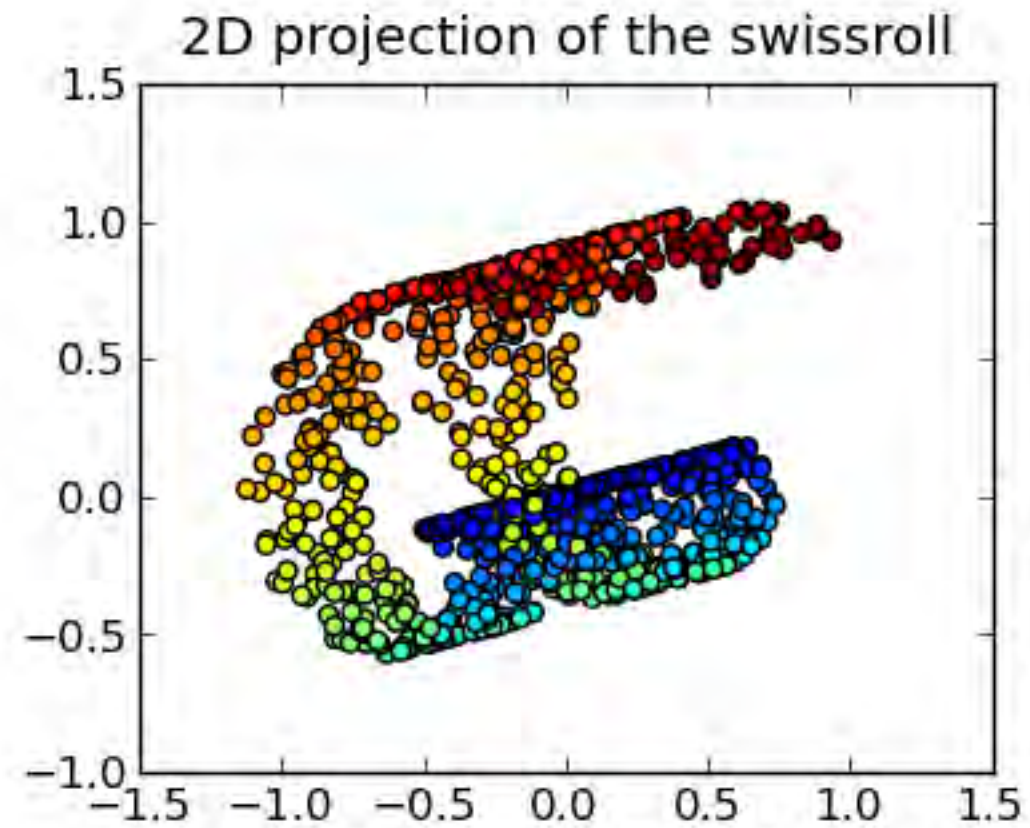


Unsupervised Learning

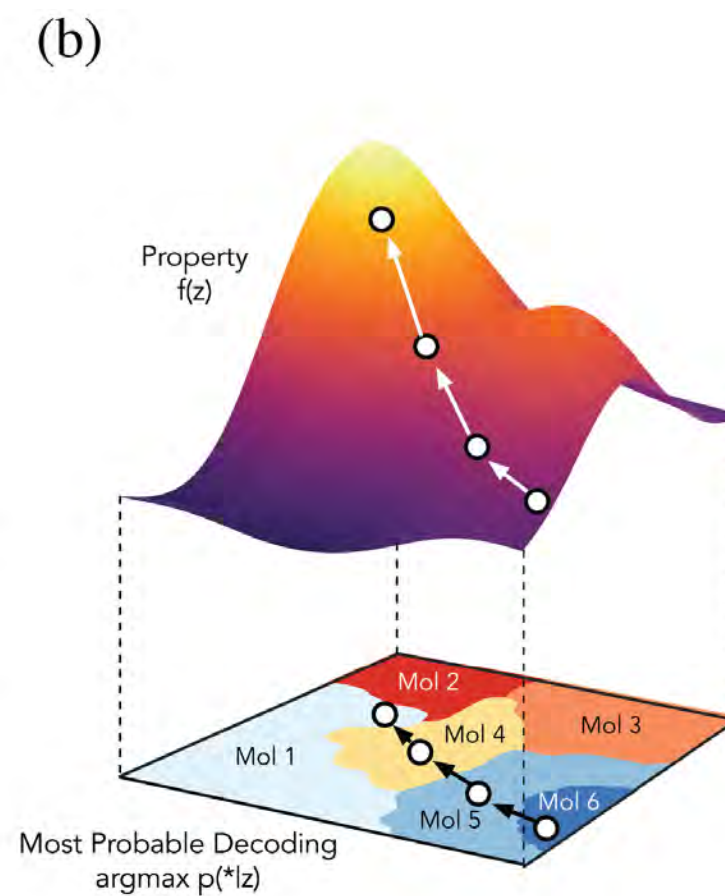
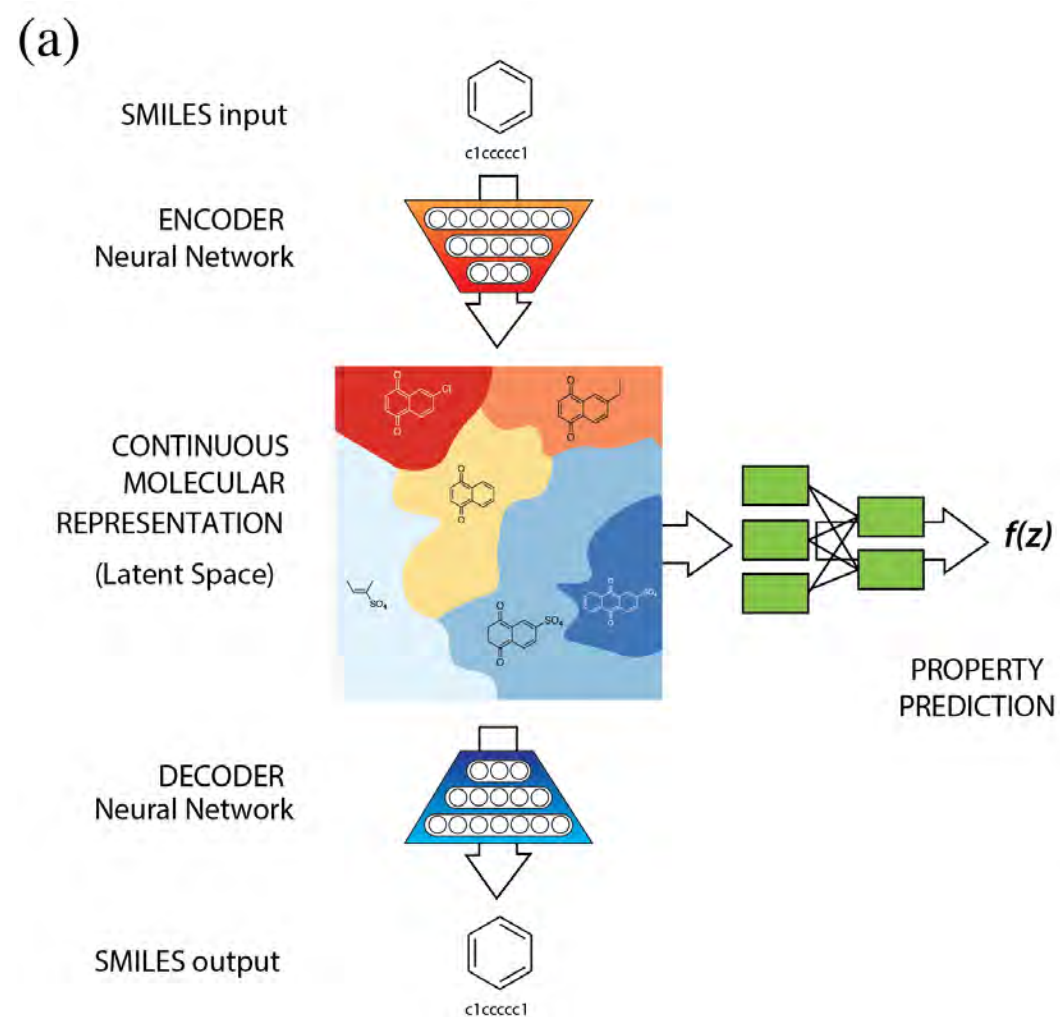
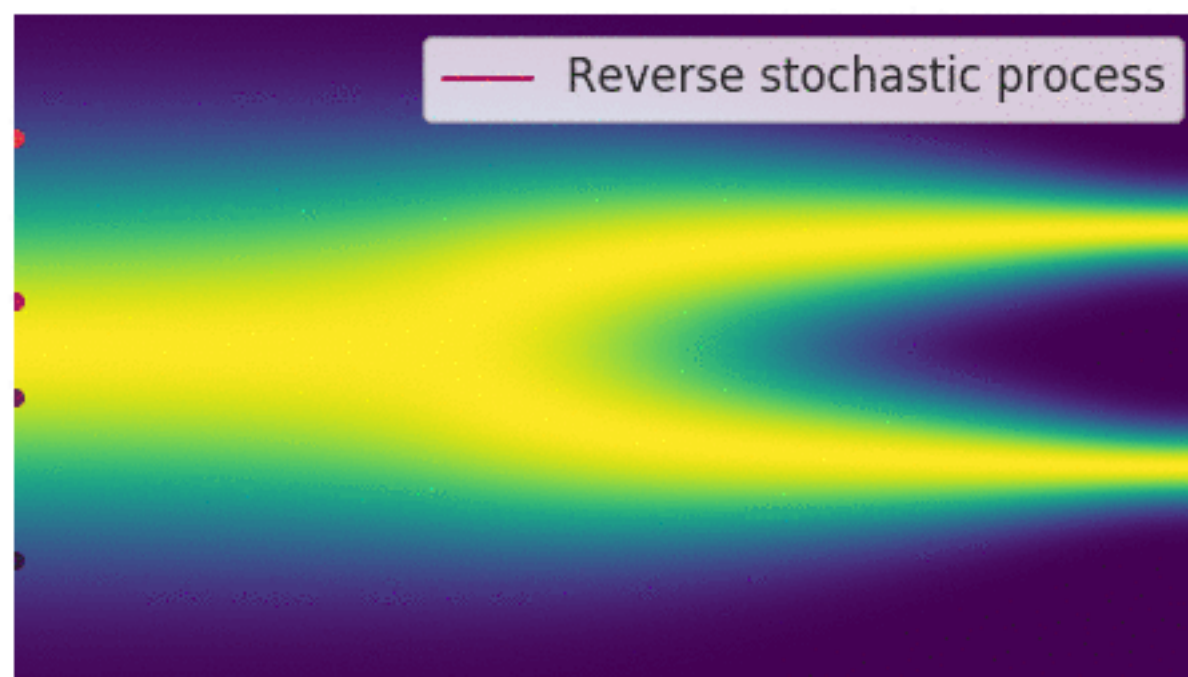
Given $\{x_i\}_{i=1}^n$, find $e : \mathcal{X} \rightarrow \mathcal{Z}$ and $d : \mathcal{Z} \rightarrow \mathcal{X}$
with $\dim \mathcal{Z} \ll \dim \mathcal{X}$ and $d(e(x)) \approx x$

or, more honestly: find a $p(x)$ you can *sample* from

Dimensionality reduction



Generative Models



Sequential Decision Making

$$x_{n+1} = \arg \max_{x \in \mathcal{X}} \alpha(x; p(f \mid \mathcal{D}_n))$$

the acquisition depends on the posterior distribution, not on a point estimate.
No uncertainty, no decision-making!

Intelligent experimentation

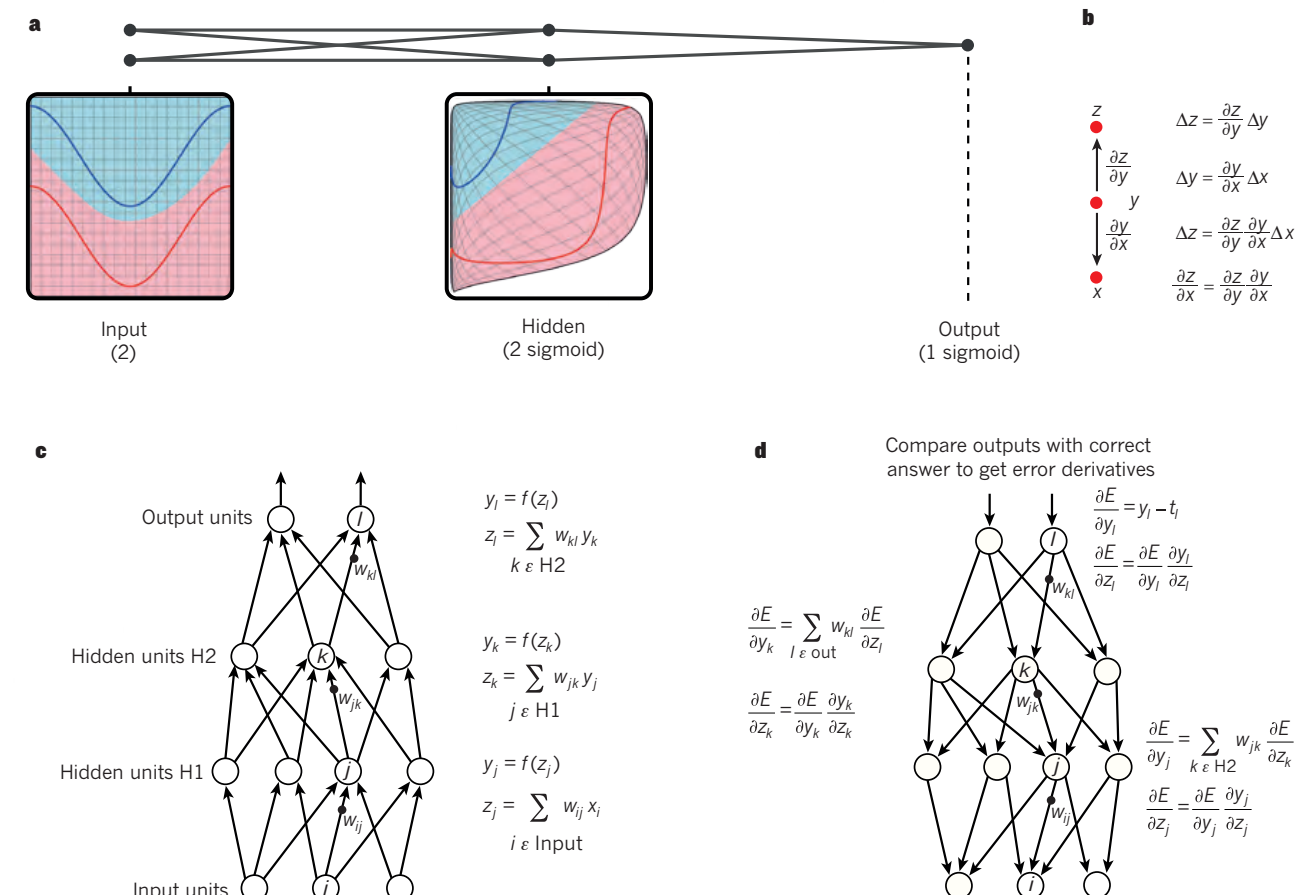
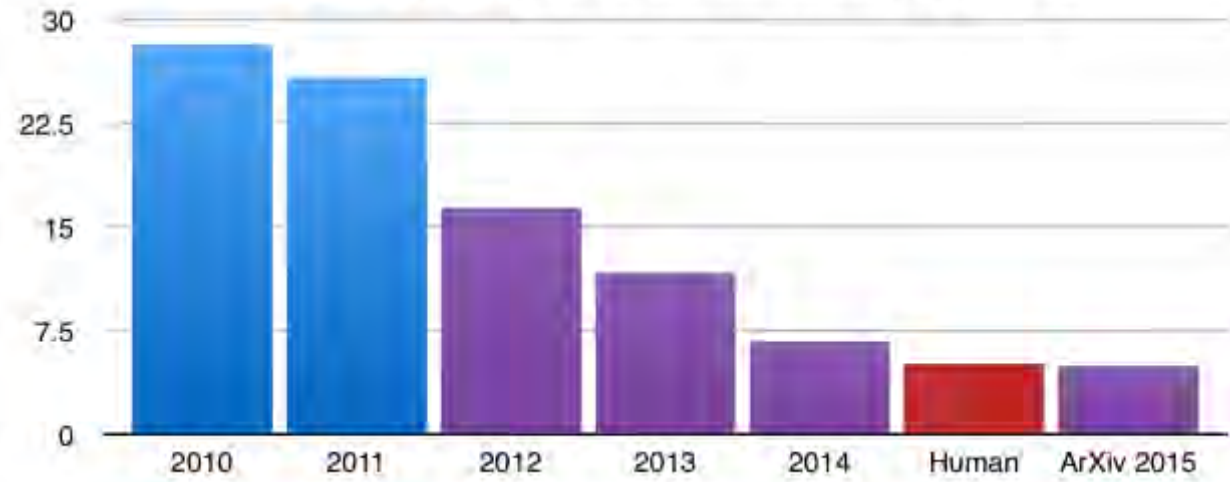


“In its first year of operation, the Intelligent Towing Tank (ITT) conducted about 100,000 total experiments, essentially completing the equivalent of a PhD student’s five years’ worth of experiments in a matter of weeks.”

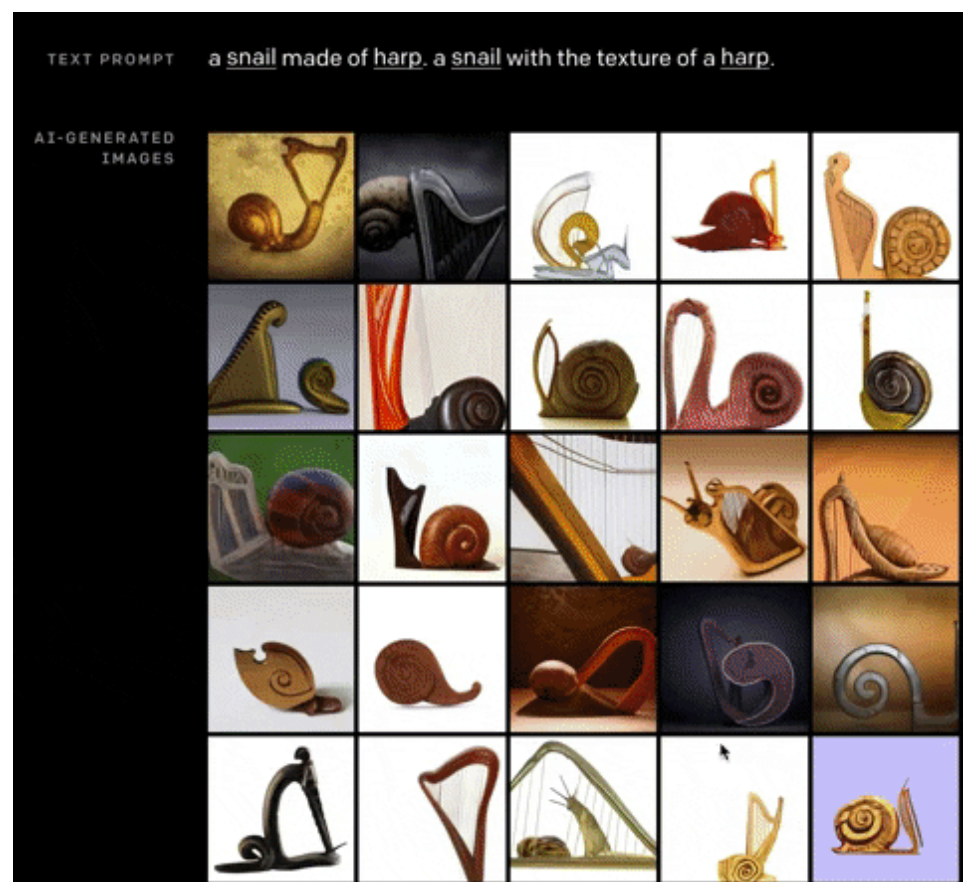
<https://news.mit.edu/2019/intelligent-towing-tank-propels-research-1209>

The Deep Learning Revolution

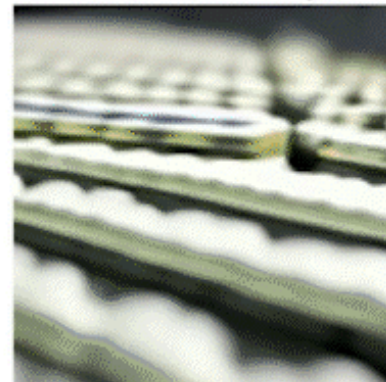
ILSVRC top-5 error on ImageNet



Working with high-dimensional data



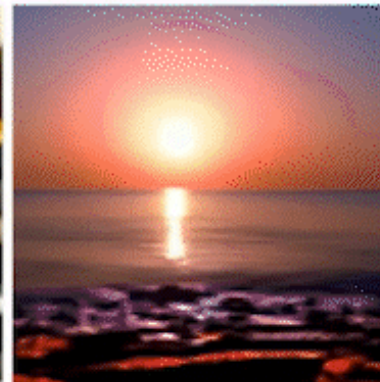
Aluminum cans on conveyor belt



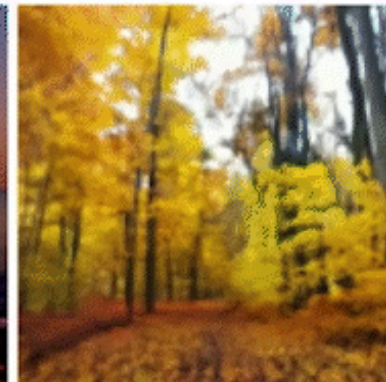
Technician concept



Dramatic ocean sunset



Forest in Autumn



Berlin - Brandenburg Gate at night



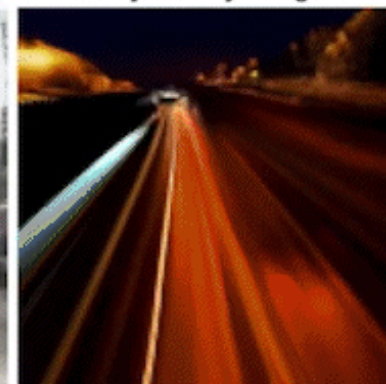
Aerial of horses on a pasture



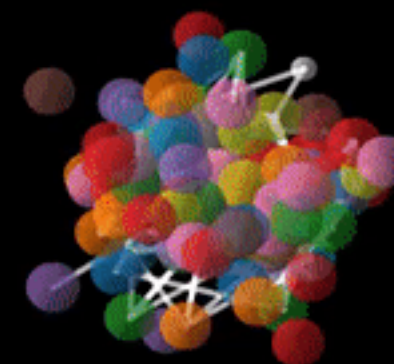
Traffic jam on 23 de Maio avenue, both directions, south of Sao Paulo,



Busy freeway at night



Denoising Molecule Diffusion



$$z_T \cdots z_t \xrightarrow{p(z_{t-1}|z_t)} z_{t-1} \cdots x, h$$

Working with multiple data modalities



draw a linear regression with python using matplotlib



Is this the right course for me?

Yes, if you:

- (a) Fulfill the pre-requisites (linear algebra, probability & statistics, optimization, Python programming).
- (b) Want to build foundational knowledge in ML for engineering research and applications.
- (c) Want to sharpen your implementation skills.

No, if you:

- (a) Do not fulfill the pre-requisites.
- (b) Are solely interested in theory.
- (c) Are solely interested in CS applications (e.g. computer vision, NLP).
- (d) Are solely interested in specialized topics (e.g. RL, geometric deep learning).

Course goals

Foundational Competencies:

- Master probabilistic modeling frameworks and Bayesian inference for scientific applications
- Implement advanced optimization algorithms including stochastic methods and second-order techniques
- Apply variational inference and MCMC sampling for complex probabilistic models
- Quantify and propagate uncertainty in computational models and data-driven predictions

Deep Learning Expertise:

- Design and implement neural networks for scientific computing applications
- Apply convolutional architectures to spatial data in engineering and physics
- Leverage Transformer models and attention mechanisms for scientific sequence modeling
- Build recurrent architectures for time-series analysis and dynamical system modeling

Physics-Informed AI:

- Develop physics-informed neural networks (PINNs) that respect physical constraints
- Implement neural operators for solving partial differential equations
- Integrate domain knowledge and conservation laws into machine learning models
- Apply Gaussian processes for multi-fidelity modeling and active learning

Generative Modeling:

- Build and train variational autoencoders for dimensionality reduction and data generation
- Implement diffusion models for high-quality sample generation
- Apply flow matching and normalizing flows for density estimation and sampling
- Design generative models for scientific data synthesis and augmentation

Systems Integration:

- Combine multiple ML techniques into coherent scientific computing pipelines
- Perform experimental design and active learning for efficient data collection
- Bridge traditional computational methods with modern AI approaches
- Deploy uncertainty-quantified ML systems for real-world engineering applications

Course logistics

- Duration: 15 Weeks
- Time: Tuesdays and Thursdays, 3:30pm to 5:00pm (AGH 105)
- Dates: August 25 to December 7, 2026
- Instructor office hours: Thursdays 12-2pm
- TA office hours: TBD
- Bi-weekly quizzes, 2 coding assignments, 2 midterms, final project

For more details visit the course website:

<https://www.seas.upenn.edu/~enm5310/>

Slides and code can be found on Github:

<https://github.com/PredictiveIntelligenceLab/ENM5310>

Tentative Syllabus

Part I: Foundations of Probabilistic Learning

Week 1: Probability theory, statistics, and information theory fundamentals

Week 2: Statistical estimation: Maximum likelihood, MAP, and Bayesian inference

Week 3: Optimization foundations: Gradients, Hessians, and stochastic optimization

Week 4: Linear models and generalized linear regression with uncertainty quantification

Part II: Advanced Inference and Sampling

Week 5: Variational inference and expectation maximization

Week 6: Markov Chain Monte Carlo and advanced sampling methods

Week 7: Midterm Exam

Part III: Deep Learning Architectures

Week 8: Multi-layer perceptrons and universal approximation theory

Week 9: Convolutional neural networks and their applications to scientific data

Week 10: Transformers and attention mechanisms for scientific computing

Week 11: Recurrent networks, LSTMs, and sequence modeling for time-series

Part IV: Physics-Informed and Scientific Machine Learning

Week 12: Physics-informed neural networks (PINNs) and neural operators

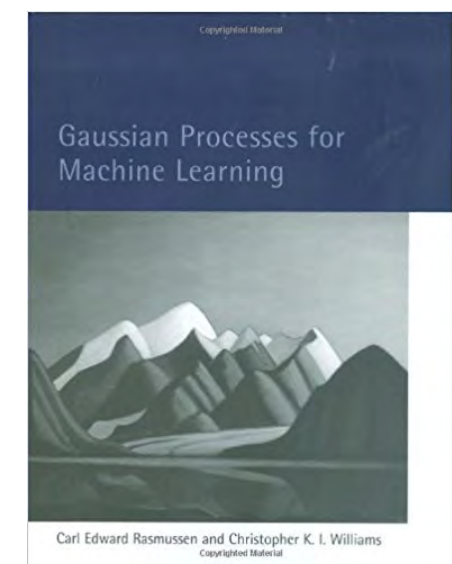
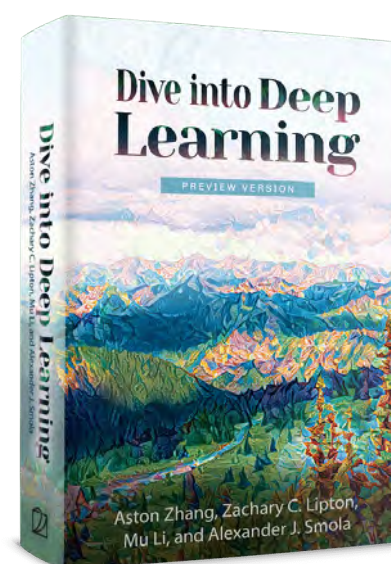
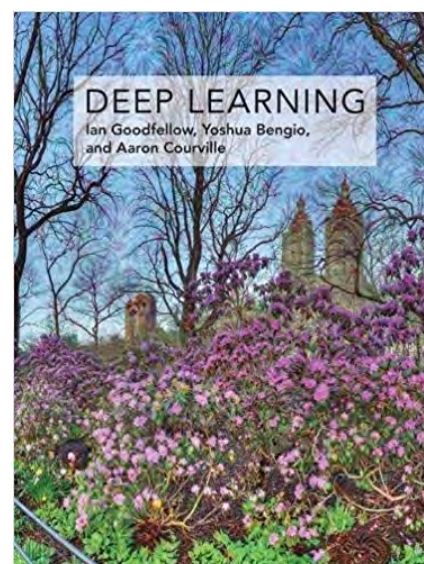
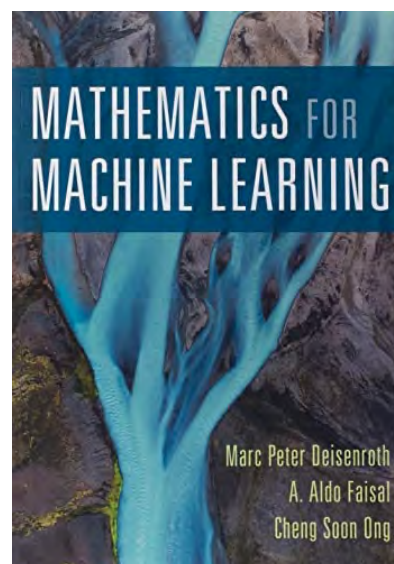
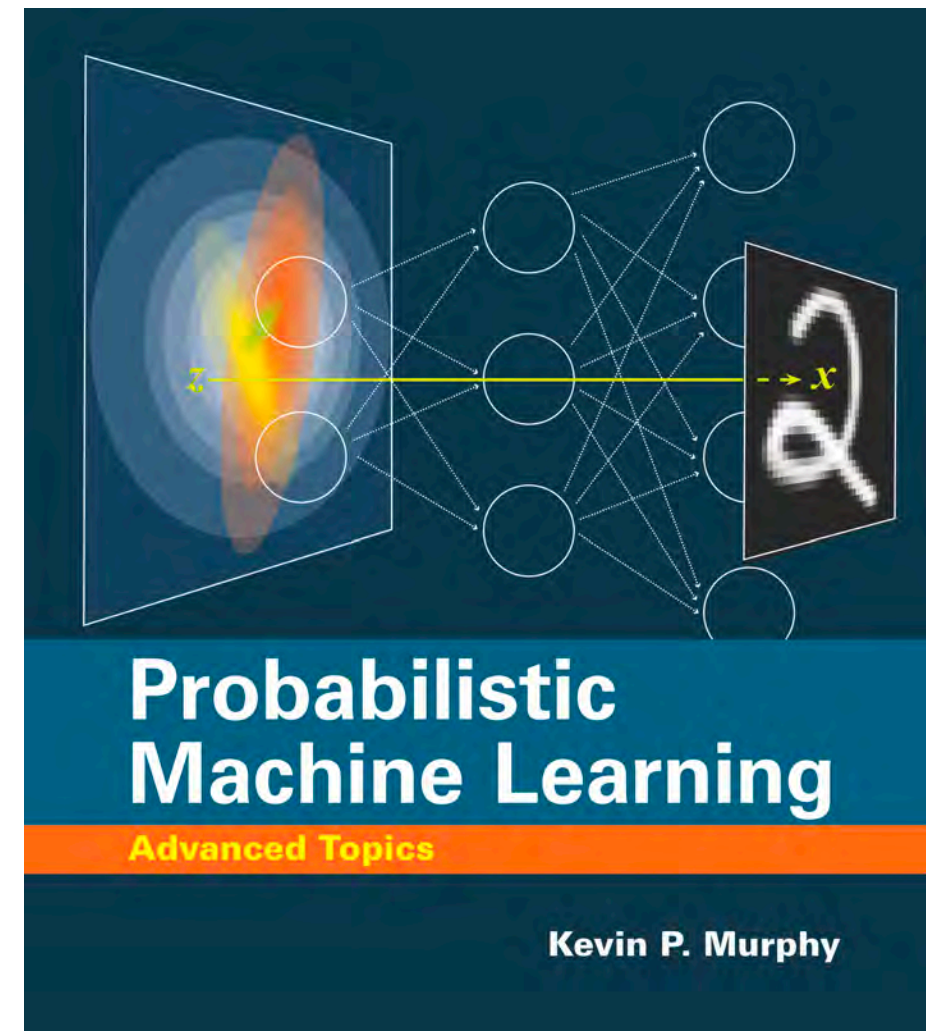
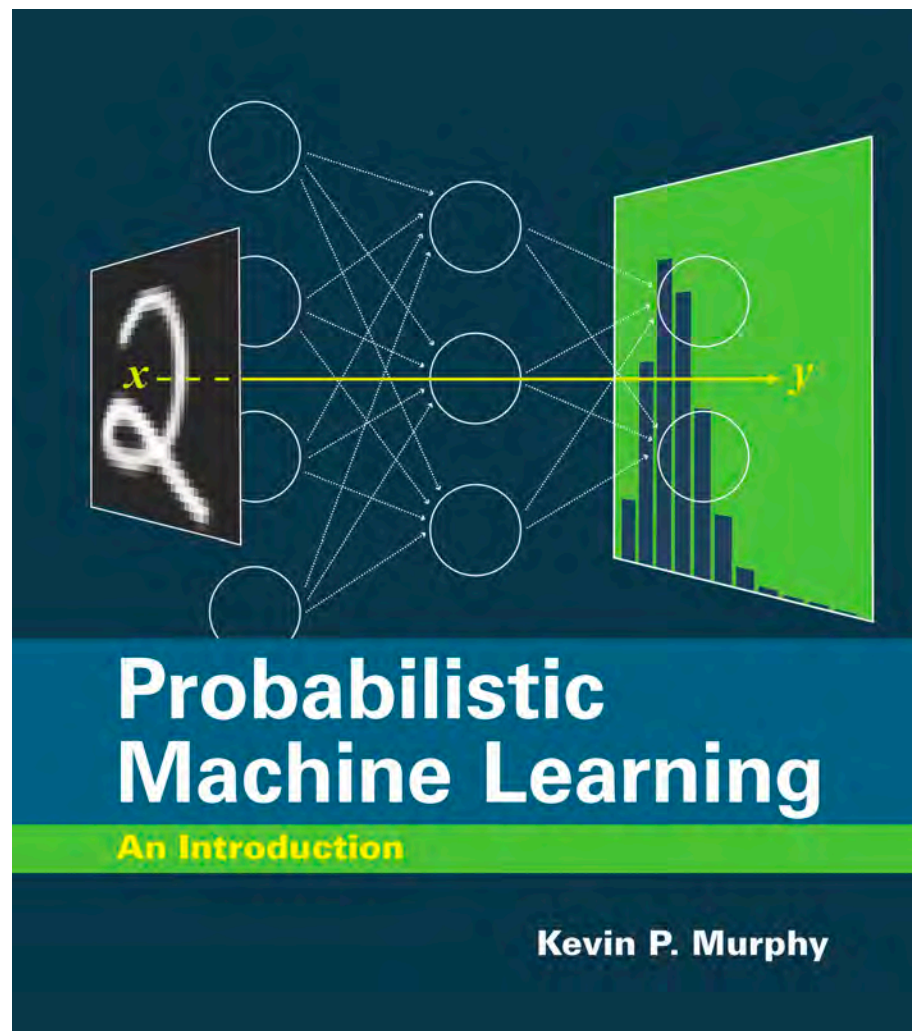
Week 13: Kernel methods, Gaussian processes, and scientific applications

Part V: Generative Models and Advanced Topics

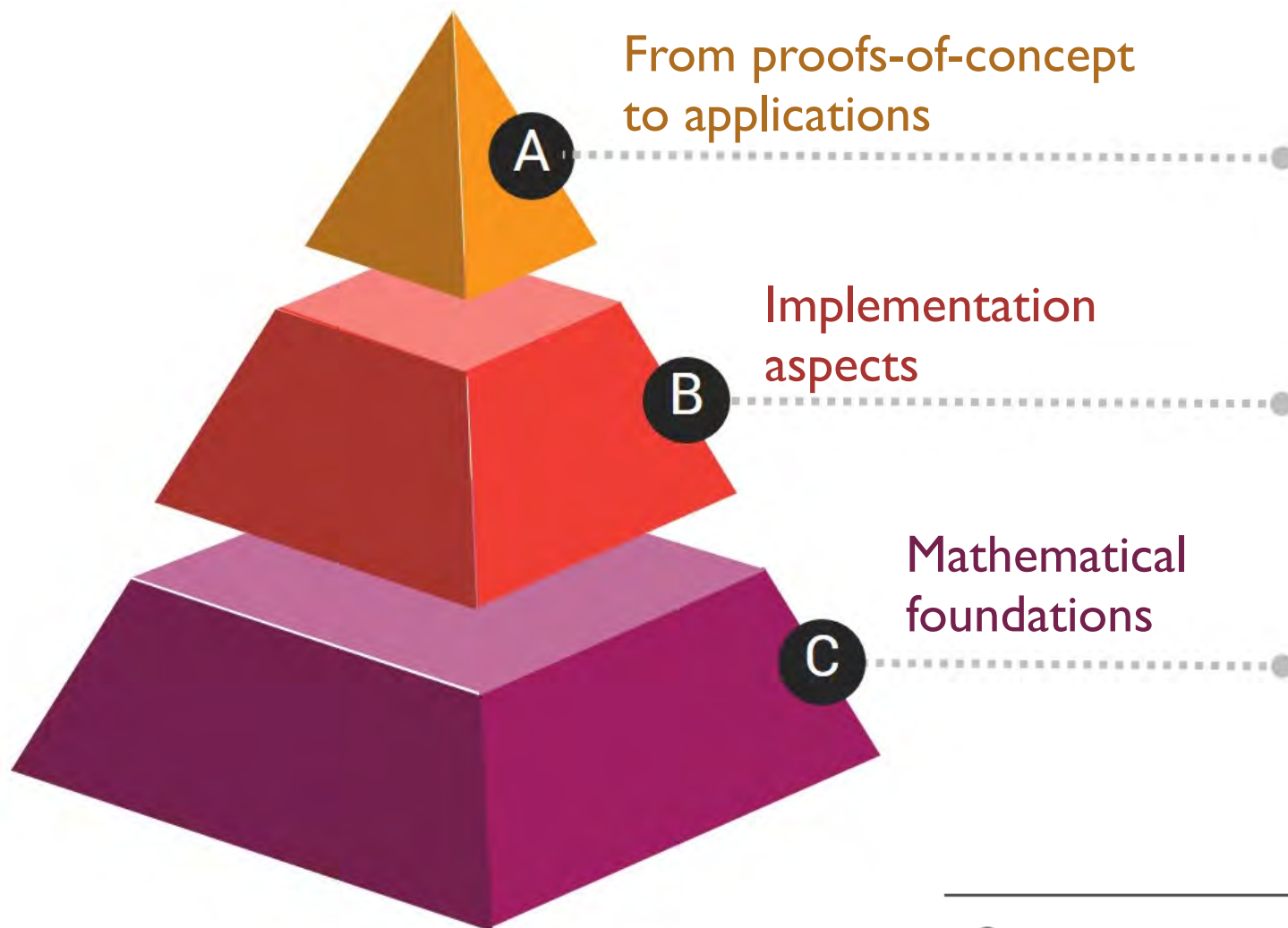
Week 14: Variational autoencoders, diffusion models, and flow matching

Week 15: Final project presentations and experimental design for ML

Textbooks



Course philosophy



Sharpen both the fundamentals as well as your computational skills. Final projects will be geared towards applications.

20% of our lecture time will be devoted on hands-on programming tutorials.

80% of our lecture time will be devoted on discussing the mathematical foundations of modern ML methods.

course evaluation plan

Component	Weight	Setting
Practice sets	ungraded	your own time
Quizzes	10%	in class, closed book
Coding assignments	30%	open-AI, + oral defense
Midterm	15%	in class, closed book
Final	15%	in class, closed book
Project	30%	open-AI, + presentation

VSCode

serviceWorker.js — create-react-app

EXPLORER

- CREATE-REACT-APP
 - .github
 - .vscode
 - node_modules
 - public
 - src
 - App.css
 - App.js
 - App.test.js
 - index.css
 - index.js
 - logo.svg
 - serviceWorker.js
 - .gitignore
 - package-lock.json
 - package.json
 - README.md
 - yarn.lock

OUTLINE

- isLocalhost
- register
 - publicUrl
 - window.a('load') callback
 - swUrl

TERMINAL

node

Compiled successfully!

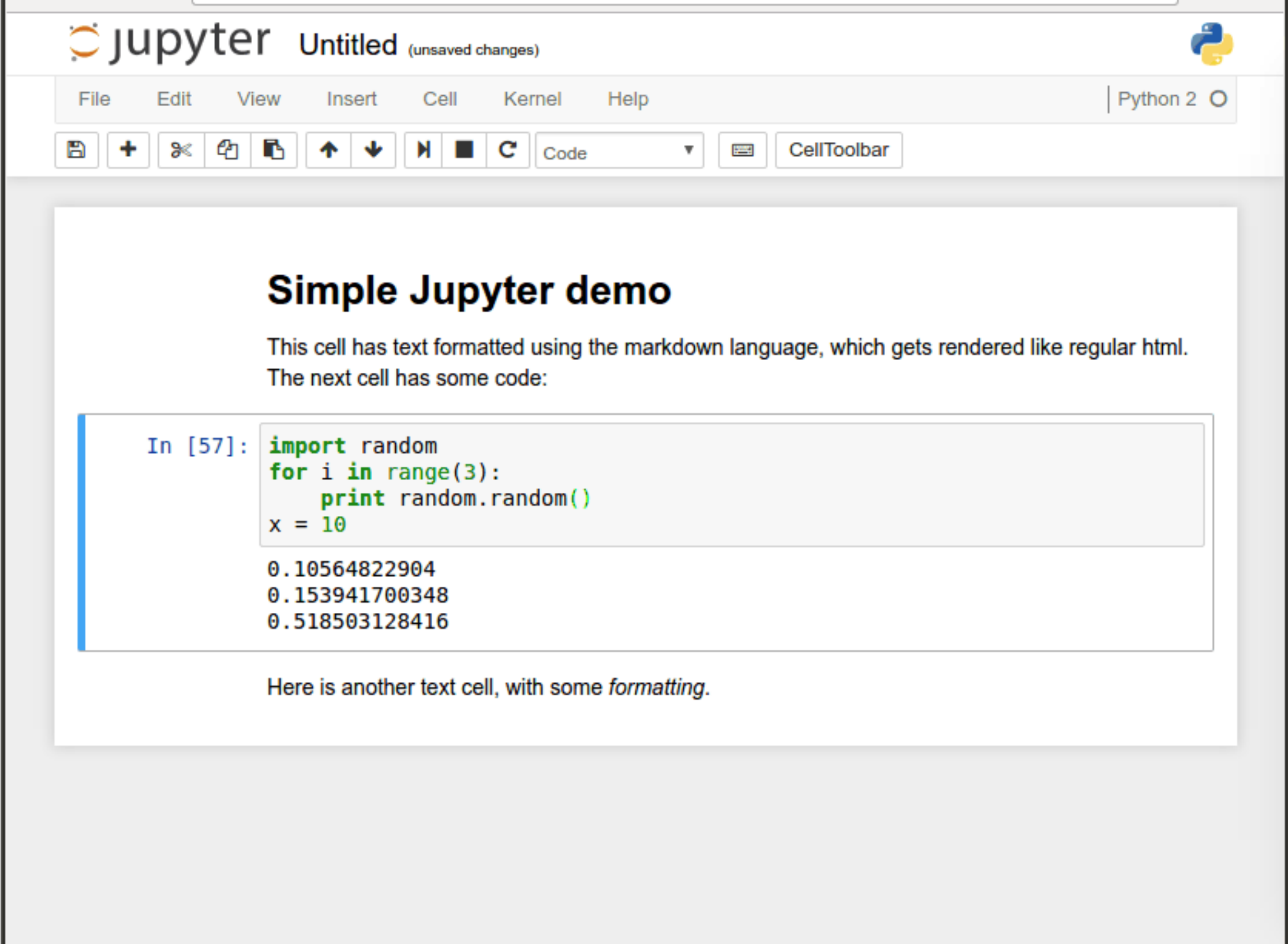
You can now view create-react-app in the browser.

Local: http://localhost:3000/
On Your Network: http://192.168.86.138:3000/

Note that the development build is not optimized.
To create a production build, use `yarn build`.

Ln 34, Col 13 Spaces: 2 UTF-8 LF JavaScript

Jupyter notebook



The screenshot shows a Jupyter Notebook window titled "Untitled (unsaved changes)". The interface includes a top menu bar with "File", "Edit", "View", "Insert", "Cell", "Kernel", and "Help". On the right of the menu bar, it says "Python 2" with a dropdown arrow. Below the menu bar is a toolbar with icons for saving, adding, deleting, copying, pasting, undo, redo, and a "Code" dropdown menu. The main content area contains two cells. The first cell is a text cell with the heading "Simple Jupyter demo" and a paragraph explaining markdown formatting. The second cell is a code cell containing Python code that imports the random module, loops three times to print random numbers, and sets a variable x to 10. The output of the code cell shows three random numbers. Below the code cell is another text cell with the text "Here is another text cell, with some *formatting*."

jupyter Untitled (unsaved changes)

File Edit View Insert Cell Kernel Help Python 2

Code CellToolbar

Simple Jupyter demo

This cell has text formatted using the markdown language, which gets rendered like regular html.
The next cell has some code:

```
In [57]: import random
for i in range(3):
    print random.random()
x = 10
```

0.10564822904
0.153941700348
0.518503128416

Here is another text cell, with some *formatting*.

Google colab

The screenshot shows a Google Colab notebook interface. The browser tabs at the top include 'face_detection - Google Drive' and 'face_detection.ipynb - Colaboratory'. The address bar shows the URL 'colab.research.google.com/drive/1ss4n...'. The notebook's left sidebar shows a file explorer with a folder named 'sample_data'. The main area of the notebook contains the following sections and code:

- Using Google Colab to run our machine learning projects.**
- Change the directory to face_detection folder**
First create a folder and name it as **face_detection**
- Code block:

```
from google.colab import drive
drive.mount('/content/drive/')
```
- Text: 'use os.chdir to connect to the face_detection directory'
- Code block:

```
from os import chdir
chdir('/content/drive/My Drive/face_detection')
```
- Clone your project in the google drive**
- Code block:

```
!git clone https://github.com/masouduut94/Pytorch_Retinaface.git
```
- Output text:

```
Cloning into 'Pytorch_Retinaface'...
remote: Enumerating objects: 26, done.
remote: Counting objects: 100% (26/26), done.
remote: Compressing objects: 100% (18/18), done.
remote: Total 146 (delta 7), reused 23 (delta 5), pack-reused 120
Receiving objects: 100% (146/146), 12.34 MiB | 24.73 MiB/s, done.
Resolving deltas: 100% (46/46), done.
```
- Text: 'Doing a little bit cleaning.'
- Code block:

```
!mv Pytorch_Retinaface/* ./
!rm -r Pytorch_Retinaface/
```
- Get the pretrained weight files**
- Text: '1 - We open the link to the pretrained models.'
- Text: '2 - We right click on the weight file and choose "Add Shortcut to Drive"'
- Text: '3 - Then we change the directory to face_detection folder and run the code below'

At the bottom left, a disk usage indicator shows '37.15 GB available'.

<https://colab.research.google.com/notebooks/intro.ipynb>